TSID

Generate sortable IDs and recover their timestamps

Install & load

```
SET allow_extension_repositories = 'allowed';
CREATE EXTENSION REPOSITORY IF NOT EXISTS query_farm WITH PREFIX 'https://duckdb-extensions.query.farm';
INSTALL tsid FROM query_farm;  LOAD tsid FROM query_farm;
```

01 At a glance

Create time-prefixed identifiers as 32-character hexadecimal strings or native UUIDs and recover their embedded timestamps. Use them as internal keys, recent-first sort values, or pagination cursors when generation time is useful alongside a compact identifier.

Generate ID → VARCHAR or UUID → Sort or decode

02 Core SQL patterns

`tsid()`

Generate a new identifier as a 32-character VARCHAR.

`tsid_uuid()`

The same identifier as a native 16-byte UUID.

`tsid(ts) / tsid_uuid(ts)`

Mint an ID for a chosen TIMESTAMP, for backfills or range bounds.

`tsid_to_timestamp(id)`

Decode a VARCHAR or UUID ID to a timezone-free UTC TIMESTAMP.

`DEFAULT tsid()`

Generate an ID when an INSERT omits the column.

`ORDER BY id DESC`

List IDs in descending time-prefix order.

`WHERE id < cursor`

Continue descending pagination from the last returned ID.

03 Keep in mind

Generation time is not event time

The embedded timestamp records ID generation. Keep a separate event timestamp if events arrive late or describe earlier activity. Concurrent generation and clock changes also mean ID order is not a global transaction-order guarantee.

04 Generate keys automatically

```
CREATE TEMP TABLE events (
  id VARCHAR DEFAULT tsid() PRIMARY KEY,
  action VARCHAR
);
INSERT INTO events(action) VALUES
  ('login'), ('click'), ('logout');
SELECT id, tsid_to_timestamp(id) AS created_utc
FROM events
ORDER BY id;
```

The default generates one ID for each inserted row.

05 Page through recent IDs

```
SELECT * FROM events
ORDER BY id DESC LIMIT 2;

SET VARIABLE cursor = (
  SELECT id FROM events
  ORDER BY id DESC LIMIT 1 OFFSET 1
);
SELECT * FROM events
WHERE id < getvariable('cursor')
ORDER BY id DESC LIMIT 2;
```

Run after the first example; the cursor comes from the previous page's last row.

06 Native UUIDs and backfills

```
SELECT tsid_uuid() AS id,
  tsid_to_timestamp(
    tsid(TIMESTAMP '2024-06-01')
  ) AS minted;
-- minted: 2024-06-01 00:00:00
```

The random tail still varies, so minted IDs are approximate range bounds.

07 In practice

Treat the format as its own type

Decode with tsid_to_timestamp: it accepts 32 hex digits or the dashed UUID form and rejects anything else. The IDs carry UUID version 4 bits, so uuid_extract_timestamp cannot decode them; do not assume ULID or other TSID compatibility. Anyone holding an ID can recover its generation time.

Full docs & examples ↗ Source ↗